

# CSE306 – Project 1

## A Monte Carlo Path Tracer

Emeric Payer

May 18, 2026

## 1 Overview

This report describes a Monte Carlo path tracer. The renderer supports:

1. diffuse and specular surfaces
2. direct illumination from a point light with hard shadows
3. indirect illumination via cosine-weighted hemisphere sampling
4. antialiasing through stochastic sub-pixel jitter
5. ray-triangle intersection on arbitrary `.obj` meshes
6. a bounding volume hierarchy (BVH) for acceleration
7. transparent surfaces with a Schlick approximation of the Fresnel term (optional)

In our code, we have the following tunable parameters whose values affect the final render:

1. image resolution  $W \times H$  ( $512 \times 512$ )
2. samples per pixel  $N$  (50) – noise decreases as  $O(1/\sqrt{N})$
3. maximum ray depth (5) – recursion cap on reflected, refracted, and indirect bounces
4. antialiasing kernel width  $\sigma$  (0.4 px) – standard deviation of the sub-pixel Gaussian jitter
5. field of view ( $60^\circ$ )
6. light intensity  $I$  ( $10^7$ )
7. gamma  $\gamma$  (2.2)
8. BVH leaf size (4 triangles) – recursion stops when a node holds at most this many triangles

All timings reported below were measured on my 12th Gen Intel(R) Core(TM) i7-12700H processor, using `g++ -O3 -fopenmp main.cpp -o main` then `time ./main`.

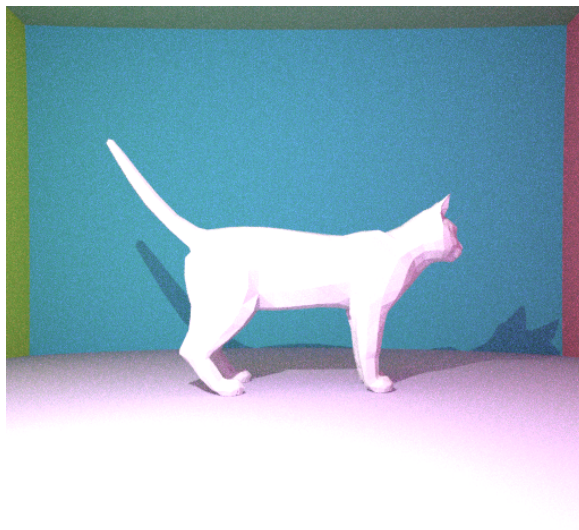
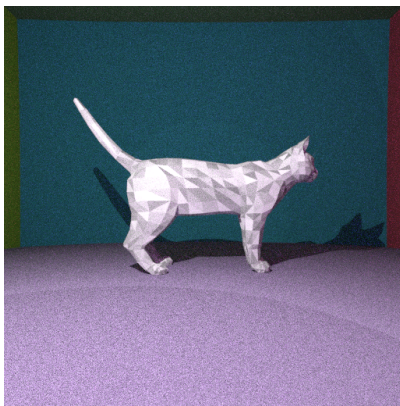


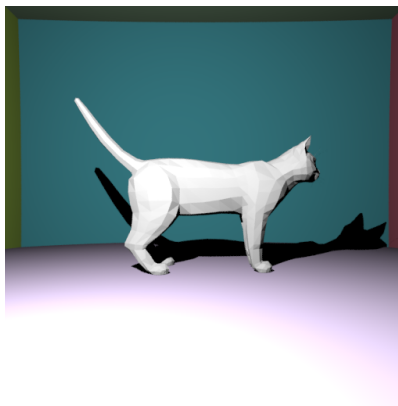
Figure 1: Final render (Render time: 10.1 seconds,  $N = 50$ )

## 2 Lab 1 (Ray casting, direct lighting, mirrors)

1. **Sphere intersection.** This is a standard quadratic solve, where we choose the smallest non-negative root ( $t_-$  when  $t_- \geq 0$ , otherwise  $t_+$ ), which correctly handles the camera-inside-sphere case (relevant since the walls are large spheres enclosing the scene).
2. **Scene traversal.** We perform a linear loop over objects keeping the closest hit. No spatial structure at the scene level since the scene only contains a handful of objects; the BVH is built per-mesh instead.
3. **Direct lighting with shadows.** The shadow rays use an offset  $\varepsilon = 10^{-5}$  along the surface normal to avoid self-intersection. This value (suggested by the TA) is the smallest that reliably eliminated shadow acne in our scenes (larger values produced light leaks near contact points).
4. **Mirror surfaces.** Recursive reflection with depth capped at `max_light_bounce` = 5. We chose 5 because most visible energy is captured within 3/4 bounces in this scene, and a hard cap avoids the cost of paths that contribute negligible radiance.
5. **Gamma correction.** Applied at output with  $\gamma = 2.2$ , as suggested. Without it, the image is noticeably underexposed in the mid-tones.



(a)  $\varepsilon = 0.0$



(b) `max_light_bounce` = 1



(c) Removing Gamma corr.

Figure 2: Effect of our parameters

## 3 Lab 2 (Indirect lighting and antialiasing)

1. **Cosine-weighted hemisphere sampling.** For each diffuse bounce we pick a random direction with probability proportional to  $\cos \theta$ , rather than uniformly. This concentrates samples where the surface actually receives energy and, conveniently, the  $\cos \theta / \pi$  pdf cancels the  $\cos \theta$  in the rendering equation and the  $1/\pi$  in the Lambertian BRDF – the indirect term reduces to `albedo * getColor(bounce)`.
2. **Local frame for the bounce.** To rotate the sample into world space we need a tangent  $T_1$  orthogonal to  $\mathbf{N}$ . We build it by zeroing the smallest-magnitude component of  $\mathbf{N}$ , which avoids the degeneracy of the more obvious construction  $T_1 = \mathbf{N} \times (0, 1, 0)$  (which fails when  $\mathbf{N}$  is near vertical).
3. **Antialiasing.** Each primary ray is offset within its pixel by a 2D Gaussian sample, generated by the Box-Muller transform. We picked  $\sigma = 0.4$  px: smaller values left aliasing on the cat silhouette, larger values blurred the image.
4. **Samples per Pixel.** We use  $N = 50$  for the final render. The figure below shows  $N = 5$ ,  $N = 50$ , and  $N = 200$ : the jump from 5 to 50 is significant, while 50 to 200 is barely visible (despite render time scaling linearly with  $N$ ).

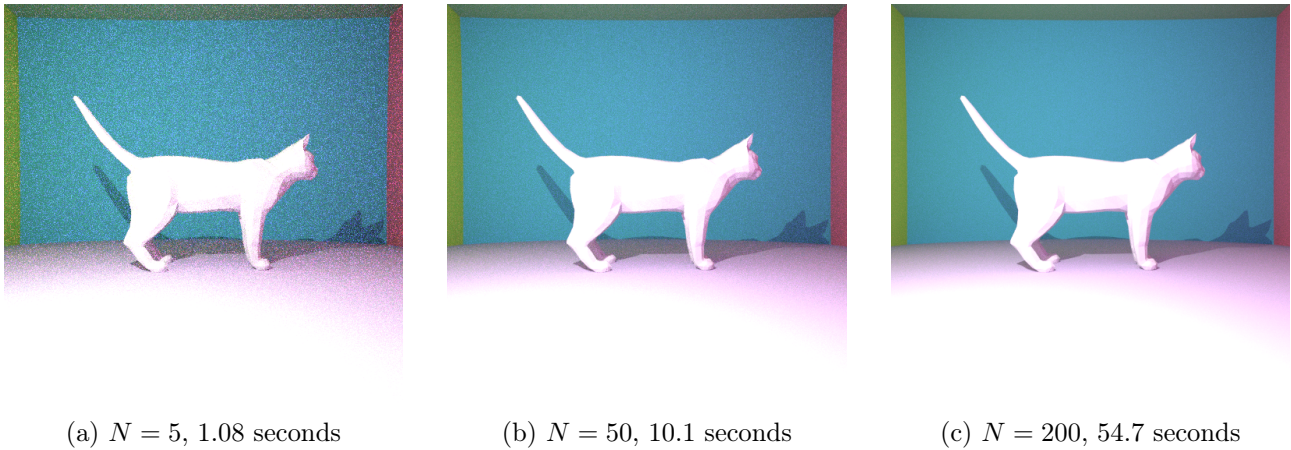


Figure 3: Effect of the Samples per Pixel ( $N$ ) parameter

## 4 Lab 3 (Ray–triangle intersection)

1. **Möller–Trumbore.** Möller–Trumbore was preferred over alternatives like Plücker coordinates because it needs no precomputed per-triangle data, keeping the mesh compact. Barycentrics are tested before  $t > 0$  since they reject most misses cheaply.
2. **Geometric (flat) normals.** The triangle’s geometric normal  $\mathbf{e}_1 \times \mathbf{e}_2$  is used directly rather than interpolating vertex normals. This produces faceted shading on the cat but keeps the intersection routine simple; smooth normals were not required.
3. **Bounding-box acceleration.** Before looping over triangles, we run a slab test on the mesh’s AABB. Most rays hit the walls, not the cat, so this single check skips the entire triangle loop for the majority of pixels (leading to a big speedup for a few lines of code).

## 5 Lab 4 (Bounding Volume Hierarchy)

The goal of this lab was to reduce render time on the mesh by replacing the brute-force per-triangle loop with a hierarchical structure.

1. **Top-down recursive construction.** Built once on the first intersection call and cached, since the mesh is static. Nodes are stored in a flat `std::vector<BVHNode>` for cache-friendly traversal.
2. **Median-index split.** We split each range at `count / 2` without sorting the triangles first. This split is also the main weakness of our BVH, since triangles aren’t ordered spatially, so the two child boxes end up overlapping, which limits how much pruning the traversal can actually do.
3. **Leaf size of 4 triangles.** Smaller leaves give a deeper tree (more traversal cost); larger leaves give more triangle tests per leaf. Four was chosen empirically; Table 2 shows that values from 2 to 8 give similar timings, while 16 starts to degrade.
4. **Traversal.** We descend the tree recursively, slab-testing each node and skipping the subtree on a miss. Both children are always visited; visiting the nearer one first would let us skip the far one once a closer hit is found, but we didn’t implement this.
5. **Degenerate-ray guard.** Rays nearly parallel to an axis are handled with a point-in-slab test on that axis instead of dividing by zero.
6. **Per-thread RNG.** We noticed that all threads were sharing `engine[0]`, which both slows things down (threads fight over the same generator) and biases the results (samples are no longer independent across threads), so we switched to `engine[omp_get_thread_num()]` so that each thread draws from its own engine, for both the indirect bounce and the antialiasing jitter.

## 5.1 Performance comparison

We compare three configurations: no acceleration (brute-force triangle loop), bounding-box only (Lab 3 final state), and BVH (this lab). All other parameters are held at the reference values:  $512 \times 512$ ,  $N = 5$  (instead of  $N = 50$  as in the final render, otherwise it takes too long), 5 bounces max.

| Acceleration      | Render time (s) | Speedup     |
|-------------------|-----------------|-------------|
| No acceleration   | <b>184.658</b>  | 1.0×        |
| Bounding box only | <b>23.095</b>   | <b>8×</b>   |
| BVH               | <b>1.055</b>    | <b>175×</b> |

Table 1: Render times under increasing acceleration

| Leaf size | Render time (s) |
|-----------|-----------------|
| 2         | <b>1.031</b>    |
| 4         | <b>1.055</b>    |
| 8         | <b>1.160</b>    |
| 16        | <b>1.402</b>    |

Table 2: Effect of BVH leaf size on render time, all other parameters fixed

## 6 Parameter variations

To illustrate the renderer’s behaviour under different conditions, we sweep three of the parameters listed in the overview around their reference values (the ones we chose for our final rendering). In each figure, the centre image is the reference render and the outer two images perturb a single parameter while holding all others fixed.

### Cat albedo

Modifying the `TriangleMesh cat(...)` constructor argument, which sets the diffuse reflectance of the mesh.

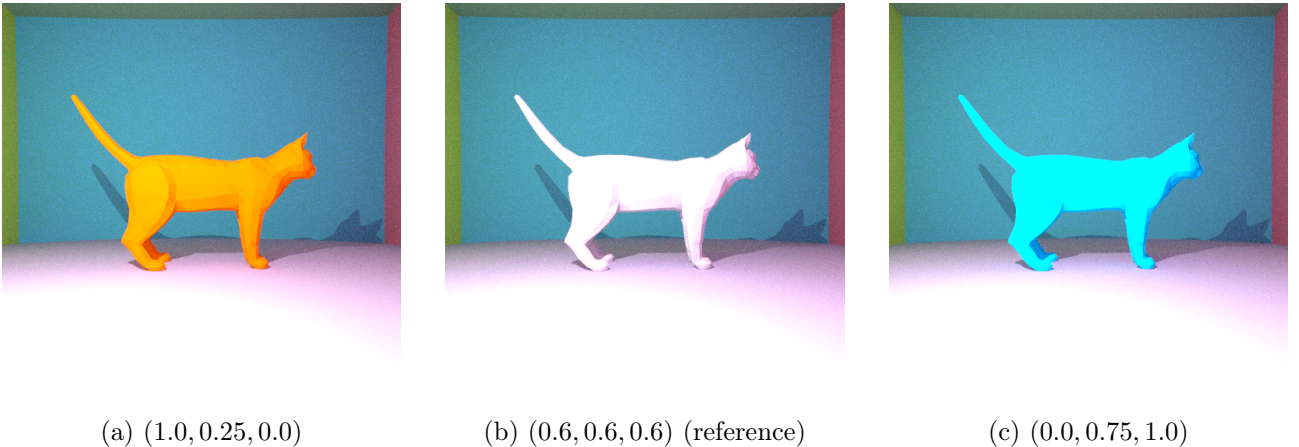
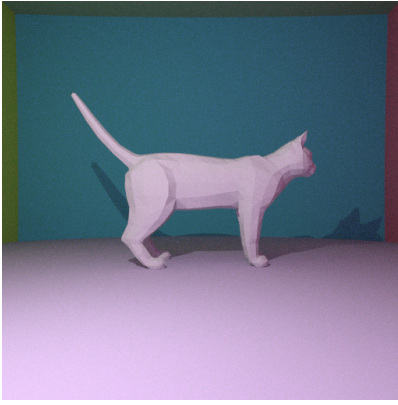


Figure 4: Varying the cat’s albedo

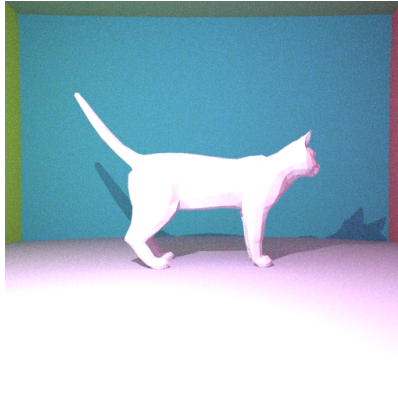
### Light intensity

Modifying `scene.light_intensity`, which scales the radiant power of the point light.

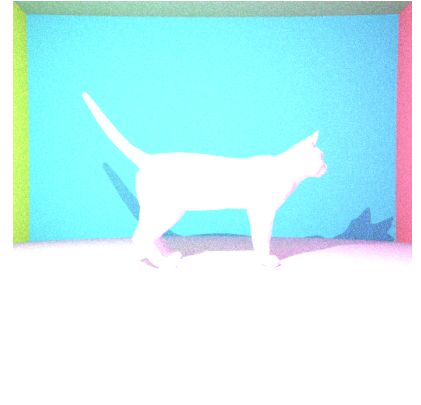




(a)  $I = 10^7$



(b)  $I = 3 \times 10^7$  (reference)

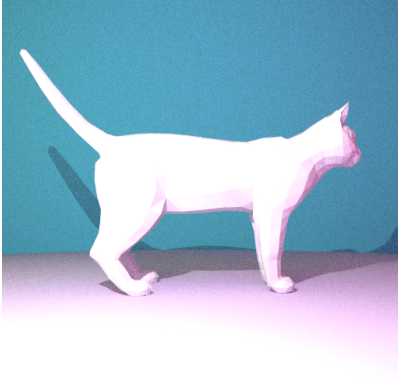


(c)  $I = 10^8$

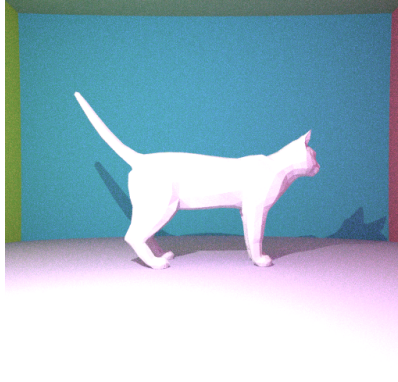
Figure 5: Varying the light intensity

## Field of view

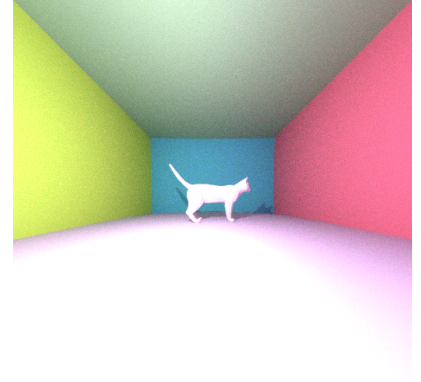
Modifying `scene.fov`, the horizontal angular extent of the image plane.



(a)  $45^\circ$



(b)  $60^\circ$  (reference)



(c)  $120^\circ$

Figure 6: Varying the field of view

## 7 Conclusion

The path tracer in its final form renders the required cat-in-Cornell-box scene at 50 spp in 10.1 seconds on my 12th Gen Intel(R) Core(TM) i7-12700H processor, illustrating diffuse and specular shading, indirect illumination, shadows, antialiasing, BVH-accelerated mesh intersection, and (optionally) refractive transparency.

In absolute terms, the biggest absolute time saving came from the bounding-box test on the mesh (roughly 160 seconds out of 180), while the BVH provided the largest relative speedup on top of that ( $23\text{ s} \rightarrow 1\text{ s}$ ). The largest visual gain came from indirect illumination.